

INSTITUTO TECNOLÓGICO DE COSTA RICA

ESCUELA DE ADMINISTRACIÓN DE TECNOLOGÍAS DE INFORMACIÓN



PROYECTO #1

BASE DE DATOS DISTRIBUIDA

OPCIÓN A: BANCA/BILLETERA REGIONAL – COCKROACHDB

BASES DE DATOS AVANZADOS

PROFESOR:

MANUEL ZUMBADO CORRALES

Por:

Emmanuel Esquivel Chavarría - 2022312336

Yerik Chaves Serrano - 2022437493

Jose Pablo Guerrero Duarte - 2022099311

SEPTIEMBRE, 2026

1 Requisitos y regiones

Este informe documenta el Proyecto 1 completo: el diseño de fragmentación y asignación (E1), su implementación sobre un clúster de tres nodos (E2), la medición de latencia local contra latencia que cruza región (E3), la prueba de falla de sitio (E4) y la crítica final sobre si la distribución estaba justificada (E5). Cada número que aparece en las secciones 4 y 5 proviene de un archivo de evidencia generado por los programas del equipo, listados en el Apéndice B.

De las tres opciones de dominio propuestas en la guía, el equipo escogió la **Opción A: banca / billetera regional**, porque su regla de residencia (los datos personales del cliente deben permanecer en la región donde abrió su cuenta) se resuelve con la misma fragmentación horizontal primaria que se aplica al resto de las tablas, sin necesidad de reglas de negocio adicionales como las que exigen la Opción B (sincronización batch desde un centro de distribución) o la Opción C (modelado de consentimiento para cruzar frontera de hospital). El sistema simula tres regiones, cada una atendida por un nodo distinto del clúster de CockroachDB. La Tabla 1 describe esa correspondencia, que es la base de todo lo que sigue: define qué significa **local** y qué significa **cruzar región** en las mediciones de la Sección 4.

Región	Nodo	Contenedor	Rol en el proyecto
cr-sj	n1	ti4601-crdB-1	Nodo de entrada (gateway) fijo de todos los programas cliente; región de referencia R1 en las mediciones.
cr-limon	n2	ti4601-crdB-2	Región remota R2 en las mediciones; es el nodo que se detiene en la prueba de falla (E4).
us-east	n3	ti4601-crdB-3	Tercera región; aporta el tercer voto del quórum Raft y aloja su propio fragmento.

Cuadro 1: Correspondencia entre regiones simuladas, nodos y contenedores del clúster.

El motor es **CockroachDB CCL v23.2.31** (imagen oficial de Docker, etiqueta `latest-v23.2`), desplegado con Docker Compose, y el cliente es **Python con psycopg**, tal como exige el stack obligatorio del enunciado. Las tres regiones son localidades

lógicas declaradas con `-locality`, ejecutadas sobre una sola computadora; esta limitación se declara explícitamente aquí porque condiciona la interpretación de todas las mediciones de la Sección 4.

2 Diseño E1: fragmentación y asignación

2.1 Esquema lógico (llaves primarias y foráneas)

El esquema lógico se compone de cuatro relaciones. La tabla **cliente** concentra la información personal identificable (PII) del titular, entre ella su nombre y su número de documento, además de la columna **region_apertura**, la cual determina en qué región reside dicho cliente de acuerdo con la regla de residencia establecida. La tabla **cuenta** está relacionada con **cliente** mediante una llave foránea y contiene, entre otras columnas, el saldo y la región en la que la cuenta fue abierta. La tabla **movimiento** registra cada transacción (depósito, retiro o transferencia) asociada a una cuenta específica. Por último, **catalogo_region** es una tabla de referencia, prácticamente estática, que no participa de la fragmentación sino de la réplica, según se explica en la Sección 2.4.

Tabla	Llave primaria	Llave foránea	Otras columnas
catalogo_region	codigo_region	(ninguna)	nombre, pais
cliente	id_cliente	region_apertura catalogo_region (codigo_region)	→ nombre, documento, correo, telefono
cuenta	num_cuenta	id_cliente → cliente (id_cliente) region → catalogo_region (codigo_region)	tipo_cuenta, saldo, fecha_apertura
movimiento	id_movimiento	num_cuenta → cuenta (num_cuenta)	tipo_movimiento, monto, fecha, region

Cuadro 2: Esquema lógico de la Opción A (llaves primarias y foráneas).

Un cliente puede poseer una o varias cuentas (relación 1:N entre **cliente** y **cuenta**), y a su vez una cuenta puede tener cero o varios movimientos asociados (relación 1:N entre **cuenta**

y movimiento); no existen relaciones de muchos a muchos dentro de este subconjunto del dominio.

La tabla `movimiento` no posee una llave foránea directa hacia `catalogo_region`, puesto que su región se deriva de la cuenta a la que pertenece, a través de `num_cuenta`. Para que el motor pueda particionar `movimiento` por región sin resolver un `join` contra `cuenta` en cada escritura, se denormalizó la columna `region` directamente en `movimiento`, de modo que la aplicación la fije con el mismo valor que `cuenta.region` al insertar el registro. Este mecanismo es lo que el enunciado denomina **fragmentación horizontal derivada**; en la Sección 3.2 se documenta cómo la implementación convirtió ese invariante, que en el diseño era una convención de la aplicación, en una restricción verificada por el motor.

2.2 Fragmentación horizontal: predicados

Las tres tablas del núcleo transaccional (`cuenta`, `cliente` y `movimiento`) se fragmentan horizontalmente usando como criterio la región de la operación. Las tablas `cuenta` y `cliente` se fragmentan de forma primaria, dado que el predicado se evalúa sobre una columna propia de cada una, mientras que `movimiento` se fragmenta de forma derivada, ya que su predicado depende de la región de la cuenta referenciada.

Tabla	Tipo	Predicado (lenguaje natural)	Condición (SQL) por fragmento
<code>cuenta</code>	Horizontal primaria	Una cuenta pertenece al fragmento de la región en la que fue abierta.	<code>region = 'cr-sj'</code> <code>region = 'cr-limon'</code> <code>region = 'us-east'</code>
<code>cliente</code>	Horizontal primaria	Un cliente reside en el fragmento de la región en la que abrió su cuenta (<code>region_apertura</code>); únicamente ahí se almacena su información personal.	<code>region_apertura = 'cr-sj'</code> <code>region_apertura = 'cr-limon'</code> <code>region_apertura = 'us-east'</code>
<code>movimiento</code>	Horizontal derivada (a partir de <code>cuenta</code> mediante <code>num_cuenta</code>)	Un movimiento pertenece al fragmento de la región de la cuenta a la que está asociado.	<code>region = 'cr-sj'</code> <code>region = 'cr-limon'</code> <code>region = 'us-east'</code>

Cuadro 3: Predicado en lenguaje natural y condición SQL de cada fragmentación horizontal.

La reconstrucción de cada tabla se obtiene mediante la unión de sus fragmentos regionales

(`UNION ALL` de `cr-sj`, `cr-limon` y `us-east`), puesto que los tres comparten exactamente el mismo esquema de columnas; en el caso de `movimiento`, la condición de fragmentación también puede expresarse como `num_cuenta IN (SELECT num_cuenta FROM cuenta WHERE region = '...')`, lo cual evidencia que dicha fragmentación se deriva de `cuenta`.

2.3 Propiedades de fragmentación

Propiedad	¿Se cumple?	Justificación
Compleitud	Sí	Las columnas <code>region</code> (en <code>cuenta</code> y <code>movimiento</code>) y <code>region_apertura</code> (en <code>cliente</code>) se definen como <code>NOT NULL</code> y restringidas al dominio <code>{cr-sj, cr-limon, us-east}</code> . En consecuencia, toda fila de las tres tablas cae necesariamente dentro de alguno de los tres fragmentos, sin que existan valores fuera de las tres condiciones definidas.
Reconstrucción	Sí	Al tratarse de una fragmentación horizontal (y no vertical), los tres fragmentos regionales de cada tabla comparten exactamente el mismo esquema de columnas. Por ello, la tabla original puede reconstruirse mediante la unión (<code>UNION ALL</code>) de sus tres fragmentos, sin que se pierda ni se duplique ninguna columna.
Disyunción	Sí	Las columnas <code>region</code> y <code>region_apertura</code> almacenan un único valor por fila (no un conjunto de valores) y están restringidas a exactamente uno de los tres códigos de región. Por lo tanto, las condiciones <code>region = 'cr-sj'</code> , <code>region = 'cr-limon'</code> y <code>region = 'us-east'</code> son mutuamente excluyentes, de modo que ninguna fila puede satisfacer dos condiciones simultáneamente y los fragmentos no se traslapan.

Cuadro 4: Verificación de las propiedades de completitud, reconstrucción y disyunción.

En la implementación, la restricción del dominio de valores no se declaró con un `CHECK` explícito, sino mediante el tipo enumerado que el propio motor genera al configurar las regiones (Sección 3.2); el efecto sobre la completitud es el mismo y además es el motor quien

lo garantiza.

2.4 Réplica

La única tabla que se replica es `catalogo_region`, con un factor de réplica igual a tres (una copia en cada uno de los tres nodos). El resto de las tablas (`cuenta`, `cliente` y `movimiento`) no se replican fuera de su región de origen, ya que hacerlo violaría la regla de residencia del enunciado e incrementaría la latencia observada sin beneficio real, dado que cada región únicamente necesita sus propios datos para operar de manera local.

Tabla / frag- mento	Factor de ré- plica	Criterio de costo
<code>catalogo_region</code>	3 (todas las re- giones)	Es una tabla prácticamente estática (pocas fi- las y una tasa de escritura casi nula) que, en cambio, se lee con frecuencia desde las tres re- giones, tanto para validar el código de región en cada alta de cliente o cuenta como para pre- sentar el nombre de la región en reportes. Re- plicarla evita resolver una lectura remota en cada operación local, a un costo de almacena- miento despreciable y sin apenas sobrecarga de sincronización, dado que su contenido casi nun- ca cambia.
<code>cuenta</code> , <code>cliente</code> , <code>movimiento</code>	1 (no se repli- can fuera de su región)	Son tablas de escritura frecuente y contienen información sensible (PII y saldos). Replicar- las fuera de su región no solo incumpliría la regla de residencia, sino que obligaría a coordi- nar el consenso entre nodos de distintas regio- nes en cada escritura, aumentando el número de <code>round-trips</code> y, con ello, el p99 observado.

Cuadro 5: Estrategia de réplica y su criterio de costo.

2.5 Asignación de fragmentos a nodos y regiones

La Tabla 6 describe qué fragmentos y qué réplicas residen en cada nodo, y la Figura 1 ilustra la asignación, incluyendo las operaciones que cruzan región (aquellas en las que el nodo de origen de la solicitud difiere del nodo donde reside el fragmento), que son exactamente las que se miden en la Sección 4.

Región / nodo	Fragmentos residentes	Qué se replica
cr-sj – Nodo n1	cuenta_cr_sj, cliente_cr_sj, movimiento_cr_sj	catalogo_region
cr-limon – Nodo n2	cuenta_cr_limon, cliente_cr_limon, movimiento_cr_limon	catalogo_region
us-east – Nodo n3	cuenta_us_east, cliente_us_east, movimiento_us_east	catalogo_region

Cuadro 6: Asignación de fragmentos y réplicas a cada nodo/región del clúster.

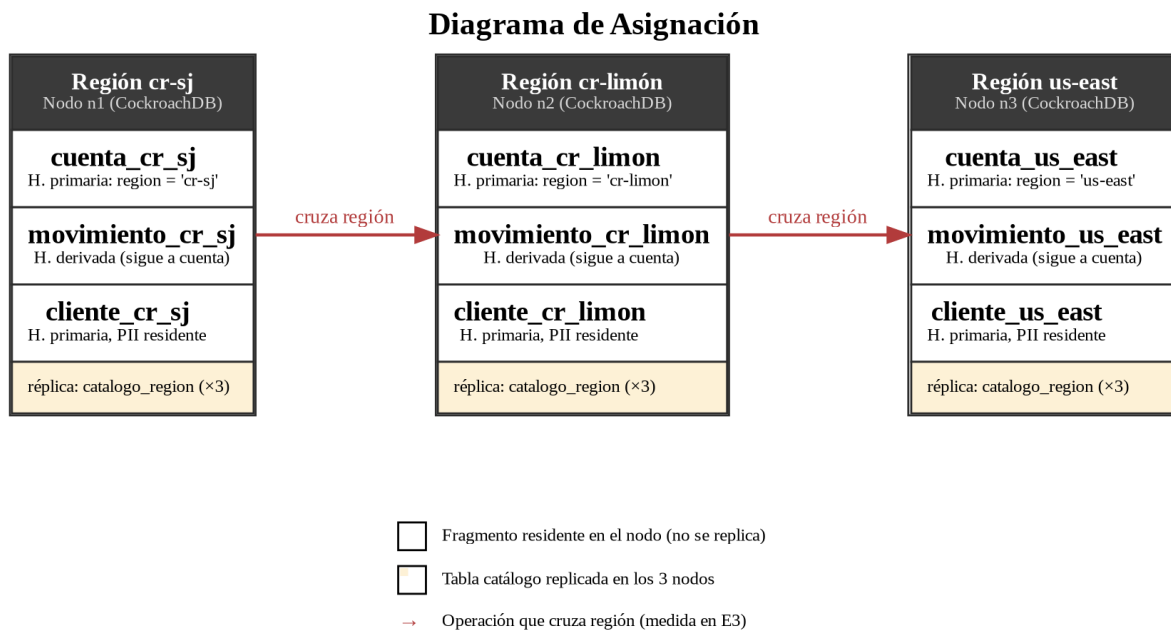


Figura 1: Diagrama de asignación de la Opción A: tres nodos de CockroachDB, uno por región, con la tabla `catalogo_region` replicada en los tres y las operaciones que cruzan región señaladas en rojo.

3 Implementación E2

3.1 Entorno del clúster

El repositorio es autocontenido, incluye su propio `docker-compose.yml`, que levanta tres servicios de CockroachDB con la misma configuración del Lab 1 (opciones `-join` compartidas entre los tres nodos y una `-locality` distinta por nodo), un servicio de un solo uso (`crdb-init`) que ejecuta `cockroach init` y crea la base `ti4601`, y un contenedor cliente (`app-crdb`) construido sobre `python:3.12-slim` con `psycopg` y `psql`. Todos los programas se ejecutan dentro de ese contenedor, nunca en el host.

Una decisión de configuración es relevante para las mediciones: el contenedor cliente fija `PGHOST=crdb-1`, es decir, **todas** las operaciones entran siempre por el nodo de `cr-sj`. Esto es deliberado, porque permite variar únicamente la región hogar de la fila consultada y atribuir la diferencia de latencia al cruce de región, y no a haber cambiado de nodo de entrada.

Antes de crear cualquier tabla, la base se convierte en multi-región. Este paso es obligatorio porque es el que genera el tipo enumerado `crdb_internal_region` que luego usan las columnas de partición.

```
ALTER DATABASE ti4601 PRIMARY REGION "cr-sj";  
ALTER DATABASE ti4601 ADD REGION "cr-limon";  
ALTER DATABASE ti4601 ADD REGION "us-east";
```

3.2 Del diseño lógico al DDL de CockroachDB

La Tabla 7 muestra cómo se tradujo cada decisión del diseño E1 a la localidad correspondiente del motor.

Tabla	Localidad aplicada	Por qué esa y no otra
<code>catalogo_region</code>	<code>LOCALITY GLOBAL</code>	Materializa el factor de réplica 3 de la Tabla 5. <code>GLOBAL</code> está pensada para tablas de lectura frecuente y escritura casi nula, y permite leerlas con latencia local desde las tres regiones sin resolver un join remoto.
<code>cliente</code>	<code>REGIONAL BY ROW AS</code> <code>region_apertura</code>	Es la traducción directa de la fragmentación horizontal primaria: cada fila tiene una región hogar determinada por su propia columna, que es justamente el predicado de la Tabla 3.
<code>cuenta</code>	<code>REGIONAL BY ROW AS</code> <code>region</code>	Misma razón que <code>cliente</code> , con la columna <code>region</code> como criterio.
<code>movimiento</code>	<code>REGIONAL BY ROW AS</code> <code>region</code>	Coloca cada movimiento en la región de su cuenta, que es el efecto buscado por la fragmentación derivada, sin que el motor tenga que consultar <code>cuenta</code> en cada escritura.

Cuadro 7: Traducción del diseño E1 a las primitivas de localidad de CockroachDB.

La implementación introdujo tres ajustes físicos con respecto a lo planeado cuando se desarrolló el entregable 1. Ninguno cambia entidades, atributos ni predicados; los tres son requisitos del motor y se documentan aquí porque el equipo debe poder sustentarlos.

1. **Tipo de la columna de región.** `codigo_region` pasa de una cadena libre al tipo enumerado `crdb_internal_region` que genera el motor al configurar las regiones. Es condición necesaria para poder declarar `REGIONAL BY ROW AS`, y como efecto adicional

el dominio de valores queda garantizado por el motor (lo que respalda la propiedad de completitud de la Tabla 4).

2. **Llave primaria compuesta.** `cuenta` y `movimiento` usan PRIMARY KEY (`region`, `llave_natural`), porque CockroachDB necesita la columna de región como prefijo del índice primario para particionarlo. La llave natural conserva su significado; `cliente` mantiene además un índice único sobre `id_cliente` solo, para que la llave foránea de `cuenta` siga siendo consecuente con lo planeado en el entregable 1.
3. **Llave foránea compuesta en movimiento.** La referencia hacia `cuenta` se declara sobre (`region`, `num_cuenta`). Con esto, el invariante que el E1 dejó como responsabilidad de la aplicación (que un movimiento viva en la misma región que su cuenta) pasa a ser una restricción que el motor rechaza si se incumple.

3.3 Carga de datos

El conjunto de datos lo genera `seed.py` con una semilla fija (42), de modo que cualquier persona que repita el procedimiento obtiene exactamente las mismas filas. El volumen se escogió para que cada fragmento tenga suficientes filas como para ocupar su propio rango y no ser un caso degenerado de una sola fila, pero sin que la carga inicial tome minutos.

Región	Clientes	Cuentas	Movimientos
cr-sj	51	75	316
cr-limon	51	83	326
us-east	51	73	295
Total	153	231	937

Cuadro 8: Volumen del conjunto de datos sintético cargado por región.

Cada región incluye además un cliente y una cuenta de prueba con identificador fijo y conocido. Existen porque los programas de medición (Sección 4) y de falla (Sección 5) necesitan repetir cientos de veces la misma operación sobre la misma fila: si la fila objetivo cambiara entre corridas, las latencias no serían comparables.

3.4 Evidencia de localidad y asignación real

La verificación automática (`verify_p1.py`) comprueba nueve condiciones de solo lectura y las nueve pasan: tres nodos vivos con sus localidades, tres regiones configuradas, la localidad correcta de las cuatro tablas, presencia de datos en las tres regiones y ausencia de movimientos cuya región no coincida con la de su cuenta.

Las cláusulas de localidad que el motor reporta con `SHOW CREATE TABLE` confirman la Tabla 7:

```
PS C:\Users\chemm\Desktop\bda proyecto> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\chemm\Desktop\bda proyecto\.venv\Scripts\Activate.ps1")
(.venv) PS C:\Users\chemm\Desktop\bda proyecto> docker compose --profile lab1 run --rm --no-deps app-crdb python3 p1-banca/scripts/verify_p1.py
Container bdaproyecto-app-crdb-run-63867ca1164d Creating
Container bdaproyecto-app-crdb-run-63867ca1164d Created
[ OK ] tres nodos/localities vivos
[ OK ] tres regiones configuradas en ti4601
[ OK ] catalogo_region es GLOBAL
[ OK ] cliente es REGIONAL BY ROW AS region_apertura
[ OK ] cuenta es REGIONAL BY ROW AS region
[ OK ] movimiento es REGIONAL BY ROW AS region
[ OK ] hay clientes en las 3 regiones
[ OK ] hay cuentas en las 3 regiones
[ OK ] todo movimiento coincide en región con su cuenta

Resultado: 9/9 verificaciones.
Esquema propio (Opción A) listo para mediciones (E3).
(.venv) PS C:\Users\chemm\Desktop\bda proyecto> █
```

Figura 2: Salida de `verify_p1.py`: las nueve verificaciones de configuración en estado correcto.

La prueba más fuerte de que la asignación diseñada se cumple en el clúster real está en `SHOW RANGES`: cada partición regional quedó en su propio rango y el titular del lease de ese rango es el nodo de su región. La Tabla 9 resume esa salida y, además, cruza el conteo de filas de cada rango contra la Tabla 8: coinciden exactamente, lo que confirma que las filas de cada región están efectivamente en el fragmento que les corresponde.

Tabla	Rango	Filas	Titular del lease	Nodo	Réplicas votantes
cuenta	79	75	region=cr-sj	1	{1,2,3}
cuenta	78	83	region=cr-limon	2	{1,2,3}
cuenta	80	73	region=us-east	3	{1,2,3}
movimiento	83	316	region=cr-sj	1	{1,2,3}
movimiento	82	326	region=cr-limon	2	{1,2,3}
movimiento	84	295	region=us-east	3	{1,2,3}

Cuadro 9: Rangos por partición regional, titular del lease y conteo de filas.

3.5 Reproducción en una máquina limpia

El procedimiento completo está documentado en el README del equipo, con guías separadas para PowerShell y para bash. Resumido, son seis pasos:

```
# 1. Levantar los 3 nodos, inicializar el cluster y construir el cliente
make lab1-up
# 2. Instalar la licencia (multi-region es funcion con licencia)
make lab1-license
# 3. Configurar regiones y aplicar el esquema (dentro de psql)
make lab1-shell
psql -X -v ON_ERROR_STOP=1 -f p1-banca/sql/00_configurar_regiones.sql
psql -X -v ON_ERROR_STOP=1 -f p1-banca/sql/01_schema.sql
# 4. Cargar el dataset sintetico
python3 p1-banca/scripts/seed.py
# 5. Verificar la configuracion (meta: 9/9)
python3 p1-banca/scripts/verify_p1.py
# 6. Repetir las mediciones de la Seccion 4
python3 p1-banca/scripts/measure_latency.py --runs 50 --warmup 5
```

3.6 Límite declarado de la implementación

En la Tabla 9 se observa que las réplicas votantes de todos los rangos son los tres nodos. Esto ocurre porque el clúster tiene un solo nodo por región: CockroachDB no puede colocar la mayoría de votantes dentro de una sola región y los distribuye entre los tres nodos disponibles. Por esto, REGIONAL BY ROW garantiza aquí la **región hogar y la colocación del titular del lease**, que es lo que evidencia la Tabla 9, pero no permite afirmar que una copia física de la PII nunca sale de su región.

4 Mediciones E3

4.1 Metodología

Se define **lectura o escritura local** como aquella en la que el nodo de entrada (`crdb-1`, región `cr-sj`) y la región hogar de la fila coinciden, y **operación que cruza región** como aquella en la que el nodo de entrada sigue siendo `crdb-1` pero la región hogar de la fila es `cr-limon`, de modo que el motor debe resolver la operación contra el titular del lease ubicado en otro nodo.

El resto de las condiciones del experimento son las siguientes:

- **Perfil de recursos.** Las cuatro mediciones se tomaron en la misma corrida, sobre la misma máquina, con el laboratorio completo (los tres nodos activos) y sin cambiar límites de CPU ni de memoria entre casos.
- **Operaciones medidas.** Lectura: `SELECT saldo, tipo_cuenta FROM cuenta` filtrando por región y número de cuenta. Escritura: `UPDATE cuenta SET saldo = saldo + 0.01` sobre esa misma fila. Son representativas del dominio (consulta de saldo y afectación de saldo) y tocan la tabla fragmentada, no el catálogo replicado.
- **Descarte de arranque en frío.** Se ejecutan 5 corridas de calentamiento por caso, cuyas latencias se descartan, antes de empezar a registrar muestras.
- **Tamaño de muestra.** 50 corridas por caso (el mínimo exigido es 30), es decir 200 observaciones registradas en el archivo de muestras crudas.
- **Estadísticos.** Se reporta la mediana (p50) y el percentil 99 calculado por rango más cercano, no el promedio.
- **Latencia inyectada.** Ninguna. No se usó `tc` ni un proxy de red, por lo que no hay un valor de retardo artificial que citar.

4.2 Resultados

Caso	Desde / hacia	p50 (ms)	p99 (ms)	n	Notas
Lectura local	R1 (cr-sj)	0.413	0.604	50	Fila servida por el nodo de entrada.
Lectura remota	R1 → R2 (cr-limon)	0.635	1.174	50	Se reenvía al titular del lease en n2.
Escritura local	R1 (cr-sj)	2.500	4.120	50	Incluye consenso Raft.
Escritura que cruza región	R1 → R2 (cr-limon)	3.045	4.656	50	Consenso más salto al titular del lease.

Cuadro 10: Latencia p50 y p99 de los cuatro casos exigidos (5 corridas de calentamiento descartadas por caso).

Operación	Δ p50 (ms)	Δ p50 (%)	Δ p99 (ms)	Δ p99 (%)
Lectura (remota vs local)	0.222	53.8	0.570	94.4
Escritura (cruza región vs local)	0.545	21.8	0.536	13.0

Cuadro 11: Costo medido de cruzar región, calculado a partir de la Tabla 10.

4.3 Interpretación

Cruzar región cuesta, pero cuesta más en escritura que en lectura. En la mediana, una lectura remota cuesta 0.222 ms más que una local (53.8 %), mientras que una escritura que cruza región cuesta 0.545 ms más (21.8 %). En porcentaje el aumento es mayor en lectura porque su tiempo base es mucho menor, pero en tiempo absoluto la escritura paga más del doble. Esa diferencia tiene una explicación estructural: la lectura remota solo agrega un reenvío al titular del lease, mientras que la escritura remota agrega ese reenvío **y además** debe completar el consenso desde un coordinador que ya no es el nodo local.

La escritura local ya paga distribución. La escritura local (2.500 ms) cuesta 6.1 veces más que la lectura local (0.413 ms) pese a afectar la misma fila. Esa diferencia no se debe al cruce de región, sino al consenso Raft: como muestra la Tabla 9, cada rango tiene sus tres réplicas votantes repartidas una por región, de modo que toda escritura confirmada necesita el acuerdo de una mayoría que necesariamente atraviesa regiones. Este es el costo que el diseño paga siempre.

El p99 se degrada más que la mediana al cruzar región. En lectura, el p99 crece 94.4 % frente al 53.8 % de la mediana. Es decir, cruzar región no solo desplaza la latencia típica: alarga la cola. En un sistema real esto importa más que la mediana, porque es la cola la que define la experiencia del peor caso.

Límite de validez. Las tres regiones son localidades lógicas sobre una sola máquina y la red entre contenedores es local, por lo que las diferencias medidas están en el orden de fracciones de milisegundo. Estos números muestran correctamente que cruzar región siempre cuesta más, y la escritura sufre más que la lectura, pero no su magnitud real en una red de área amplia: en un despliegue con regiones geográficamente separadas, el salto adicional al titular del lease y el consenso pasarían a estar dominados por el tiempo de ida y vuelta de la red, del orden de decenas de milisegundos. Inferir ese costo requeriría latencia controlada, que no se usó en este experimento.

5 Falla de sitio (nodo) E4

5.1 Protocolo e instrumentación

Se siguió el protocolo del enunciado. El programa `e4_failover.py` automatiza los pasos medibles: confirma el estado sano con una escritura de prueba, repite esa misma escritura cada 0.5 s sobre la cuenta de prueba de `cr-limon`, registra cada intento con marca de tiempo, estado y latencia, y al final compara el saldo final contra el saldo esperado. La detención del nodo se hace a mano desde otra terminal, tal como pide el enunciado, para que quede como una acción explícita del operador.

La instrumentación considera un intento como caída no solo cuando devuelve error, sino también cuando confirma pero tarda más de 500 ms. Este criterio es necesario porque durante la reelección de un lease CockroachDB no siempre rechaza la escritura, con frecuencia la deja bloqueada y la confirma después, y sin ese umbral la interrupción quedaría invisible en el registro.

5.2 Caída del nodo y RTO observado

Con el clúster sano, se detuvo el contenedor del nodo de `cr-limon` mientras la sonda escribía cada 0.5 s.

Marca de tiempo (UTC)	Evento	Detalle
03:46:45.834	Estado sano	Escritura de prueba confirmada en 4.5 ms; saldo inicial 1001.10.
03:46:56.550	Último intento normal	Intento 21 confirmado en 7.8 ms.
$\approx$ 03:46:57.0	Caída del nodo	Se detiene el contenedor de <code>cr-limon</code> ; el intento 22 queda bloqueado.
03:47:01.630	Confirmación tardía	Intento 22 confirma tras 4582 ms (bloqueado durante la reelección del lease).
03:47:02.134	Servicio normalizado	Intento 23 confirma en 3.6 ms. RTO observado: 5.1 s.

Cuadro 12: Bitácora de la caída del nodo de `cr-limon`.

Una vez normalizado el servicio, las 205 escrituras restantes de la corrida se confirmaron sin ningún error y ninguna superó los 18 ms, por lo que la recuperación se mantuvo estable hasta el final de la prueba.

El RTO observado de 5.1 s corresponde al tiempo que tarda en expirar el lease del rango afectado y en completarse la reelección: hasta que ese lease no expira, ningún otro nodo puede servir esas filas, y por eso la escritura permanece bloqueada en lugar de fallar de inmediato.

5.3 RPO y quórum

Durante toda la corrida se confirmaron 228 escrituras y ninguna devolvió error. El saldo esperado tras esas 228 escrituras era 1003.38 y el saldo final leído del clúster fue exactamente 1003.38, de modo que **no se perdió ni se duplicó ninguna escritura confirmada: el RPO observado es 0**. La verificación es aritmética y no depende de interpretar el registro: cada escritura confirmada suma 0.01 al saldo, así que cualquier pérdida o duplicación habría

alterado el total.

Conviene precisar el alcance de esa afirmación. El RPO se refiere a operaciones **confirmadas**: una operación que nunca recibió confirmación tiene un resultado desconocido y no debe contarse como pérdida. En esta corrida no hubo ningún intento sin confirmación, por lo que la distinción no afectó el resultado.

El comportamiento observado se explica por el tamaño del quórum. Cada rango tiene tres réplicas votantes, una por nodo, y una escritura se confirma cuando la acepta la mayoría, es decir dos. Con un nodo caído quedan dos réplicas vivas, que siguen siendo mayoría, y por eso el servicio continúa. Con dos nodos caídos quedaría una sola réplica viva: el proceso seguiría en ejecución, pero no habría mayoría y ninguna escritura podría confirmarse.

6 Crítica E5: ¿hacía falta distribuir?

La alternativa contra la que se compara es la convencional: **un nodo primario con una réplica de lectura**, donde todas las escrituras van al primario y la réplica sirve consultas de solo lectura, sin fragmentación por región ni consenso distribuido.

Criterio	Diseño implementado (3 nodos, RBR)	Alternativa (1 primario + réplica de lectura)
Latencia de escritura	2.500 ms local y 3.045 ms cruzando región (p50). Toda escritura paga consenso de mayoría entre regiones.	Sin consenso distribuido: la escritura se confirma en el primario. Sería más barata para quien esté junto al primario y más cara para el resto, en proporción al retardo de red hacia él.
Latencia de lectura	0.413 ms local y 0.635 ms cruzando región (p50), siempre consistente.	Lectura local barata en la réplica, pero con retraso de replicación: puede devolver datos desactualizados.
Disponibilidad	Medida: RTO 5.1 s automático y RPO 0 ante la caída de un nodo, sin intervención.	La promoción de la réplica no es automática y, con replicación asíncrona, el RPO típicamente es mayor que cero.
Complejidad operativa	Alta: tres nodos, configuración multi-región, licencia, particionamiento por fila y necesidad de razonar sobre quórum.	Baja: es el despliegue estándar que cualquier equipo ya sabe operar.
Costo en la misma máquina	Tres contenedores y tres copias de cada rango.	Dos procesos y dos copias.
Residencia de datos	La región hogar y el titular del lease de cada fragmento quedan en su región (Tabla 9).	Imposible de cumplir: un primario único concentra la PII de las tres regiones.

Cuadro 13: Comparación entre el diseño implementado y la alternativa de un solo nodo primario.

Por latencia, la distribución no se justifica. Con los números medidos, distribuir no hizo nada más rápido: agregó 0.222 ms a la lectura y 0.545 ms a la escritura cuando la operación cruza región (Tabla 11) y obligó a que **toda** escritura pague un consenso que atraviesa las tres regiones, lo que se observa en que la escritura local cuesta 6.1 veces lo que la lectura local. Un primario único no pagaría ese consenso. La comparación es conservadora en contra

del diseño distribuido: aquí no hay retardo de red real, y en un despliegue geográficamente separado la alternativa haría que dos de las tres regiones pagaran el tiempo de ida y vuelta al primario en cada operación.

Por disponibilidad, la distribución aporta un beneficio comprobado. La Sección 5 lo midió: ante la caída de un nodo, el servicio se recuperó en 5.1 s por sí solo y sin perder ninguna escritura confirmada, y siguió confirmando escrituras sin error hasta el final de la prueba. La alternativa requiere detectar la falla y promover la réplica, y su RPO depende del retraso de replicación en el instante de la caída.

Por residencia, la distribución es la única opción viable. La Opción A exige que la información personal del cliente resida en la región donde abrió su cuenta. Un primario único concentra por definición los datos de las tres regiones y no puede cumplir ese requisito, sin importar cuán buena sea su latencia.

Conclusión: justificado solo por residencia, con la disponibilidad automática como beneficio secundario comprobado. Si el dominio no tuviera la regla de residencia, los números de este proyecto no alcanzarían para justificarla: el costo por operación es medible, la complejidad operativa es mayor y el beneficio de latencia es inexistente a esta escala. Es la regla de residencia, que ningún primario único puede cumplir, la que vuelve necesaria la fragmentación por región; la tolerancia a la caída de un nodo llega después, como consecuencia del mismo mecanismo de quórum.

A Repositorio y programas

Archivo	Función
<code>docker-compose.yml</code>	Define los tres nodos con sus localidades, el servicio de inicialización y el contenedor cliente.
<code>Dockerfile.app-crdb</code>	Imagen cliente: Python 3.12 con <code>psycopg</code> y <code>psql</code> .
<code>Makefile</code>	Atajos de infraestructura: <code>lab1-up</code> , <code>lab1-status</code> , <code>lab1-shell</code> , <code>lab1-license</code> , <code>lab1-down-v</code> .
<code>p1-banca/sql/00_configurar_regiones.sql</code>	Convierte <code>ti4601</code> en base multi-región.
<code>p1-banca/sql/01_schema.sql</code>	Esquema propio con las cláusulas de localidad de la Tabla 7.
<code>p1-banca/scripts/seed.py</code>	Genera y carga el dataset sintético reproducible (semilla 42).
<code>p1-banca/scripts/verify_p1.py</code>	Nueve verificaciones de solo lectura sobre la configuración.
<code>p1-banca/scripts/measure_latency.py</code>	Mide los cuatro casos de la Tabla 10.
<code>p1-banca/scripts/e4_failover.py</code>	Sonda de escritura continua, RTO y verificación de RPO.
<code>p1-banca/README.md</code>	Guía de reproducción en máquina limpia (PowerShell y bash).

Cuadro 14: Programas y archivos de configuración del equipo.

B Archivos de evidencia

Archivo	Qué respalda
<code>config-check.txt</code>	Las nueve verificaciones en estado correcto (Sección 3.4).
<code>schema-evidence.txt</code>	<code>SHOW REGIONS</code> , <code>SHOW CREATE TABLE</code> de las cuatro tablas y <code>SHOW RANGES</code> de cuenta y movimiento; es la fuente de las Tablas 7 y 9.
<code>latency.csv</code>	Las 200 observaciones crudas (4 casos $\times$ 50 corridas).
<code>latency-summary.txt</code>	Resumen p50/p99 reproducido en la Tabla 10.
<code>e4-failover-log.txt</code> , <code>e4-failover.csv</code>	Caída del nodo: bitácora, RTO de 5.1 s y verificación de RPO (Tabla 12).

Cuadro 15: Archivos de evidencia y sección del informe que respalda cada uno.

C Roles del equipo

Integrante	Carné	Responsabilidades
Emmanuel Esquivel Chavarria	2022312336	Diseño de fragmentación y asignación (E1), redacción del informe, crítica comparativa (E5), generador de datos y verificador.
Yerik Chaves Serrano	2022437493	Infraestructura del clúster (Compose, imagen cliente y Makefile), configuración multi-región, esquema con localidades, sonda y orquestación de la falla de nodo (E4).
Jose Pablo Guerrero Duarte	2022099311	Programa de medición y corridas de latencia (E3) y recolección de evidencia.

Cuadro 16: Distribución de responsabilidades dentro del equipo.